PlussQit User Guide

Practical Series/Note/Timeline/TimeIndex Documentation

PlussMa Contributors

February 27, 2026

Contents

1 Purpose **3**

2 Build and Run Demos **3**
- 2.1 Build with examples enabled . . . 3
- 2.2 Most useful demo binaries . . . 3

3 Series (CodeM) **3**
- 3.1 Include and Type Aliases . . . 3
- 3.2 Construction Cookbook . . . 4
  - 3.2.1 1) Scalars . . . 4
  - 3.2.2 2) 1D/2D/N-D from literals . . . 4
  - 3.2.3 3) From lvalue host buffers . . . 4
  - 3.2.4 4) Shape factories . . . 4
  - 3.2.5 5) Range and random factories . . . 4
  - 3.2.6 6) Like-factories . . . 4
  - 3.2.7 7) Complex factories . . . 5
- 3.3 Core Usage Patterns . . . 5
  - 3.3.1 Inspect and move data . . . 5
  - 3.3.2 Math and reductions . . . 5
  - 3.3.3 Indexing/shape transforms . . . 5
  - 3.3.4 Masking and gather/scatter . . . 5
  - 3.3.5 Signal/statistics utilities . . . 5
- 3.4 Series Demo Map . . . 6

4 Note (CodeN) **6**
- 4.1 Include and Basic Construction . . . 6
- 4.2 Factory Patterns You Will Actually Use . . . 6
- 4.3 Common Workflows . . . 6
  - 4.3.1 Text cleanup + normalization . . . 6
  - 4.3.2 Tokenization and split . . . 7
  - 4.3.3 NLP/TeX utilities . . . 7
  - 4.3.4 Printing control . . . 7
- 4.4 Note Demo Map . . . 7

5 Timeline and Duration (CodeH) 7
5.1 Construct Timeline . . . 7
5.2 Construct Duration and do date arithmetic . . . 8
5.3 Datetime field extraction, formatting, timezone . . . 8
5.4 Timeline Demo Map . . . 8

6 TimeIndex (CodeH) 8
6.1 Construct and inspect . . . 8
6.2 Resample and rolling . . . 9
6.3 Slicing, asof, set-like ops . . . 9
6.4 TimeIndex Demo Map . . . 9

7 Method Families (Quick Reference) 9
7.1 Series . . . 9
7.2 Note . . . 9
7.3 Timeline/TimeIndex . . . 10

8 Exhaustive API Index Files 10

9 Sphinx Migration Note 10

10 Build This PDF 10

1 Purpose

This guide is focused on **how to use** PlussQit in real code. It is organized in this order:

1. `Series` (CodeM)
2. `Note` (CodeN)
3. `Timeline + Duration` (CodeH)
4. `TimeIndex` (CodeH)

   For exhaustive API inventories, see Section 8.

2 Build and Run Demos

2.1 Build with examples enabled

```
cmake --preset core-dev
cmake --build --preset core-dev
```

2.2 Most useful demo binaries

- `build/PlussQit/plussqit_codem_series_usage_demo`
- `build/PlussQit/plussqit_codem_series_methods_demo`
- `build/PlussQit/plussqit_codem_stats_demo`
- `build/PlussQit/plussqit_codem_signal_demo`
- `build/PlussQit/plussqit_coden_note_factory_demo`
- `build/PlussQit/plussqit_coden_note_method_families_demo`
- `build/PlussQit/plussqit_codeh_timeline_non_generated_and_factory_methods_demo`
- `build/PlussQit/plussqit_codeh_timeline_dataframe_timeindex_methods_demo`

3 Series (CodeM)

3.1 Include and Type Aliases

```
#include <pluss/pluss.h>
// narrower:
#include <pluss/codem/codem.h>
#include <pluss/codem/series_factory.h>

using pluss::Series;
using pluss::ScalarType;
using pluss::Platform;
```

3.2 Construction Cookbook

3.2.1 1) Scalars

```
auto a = pluss::make_series(3.14);                    // scalar float
auto b = pluss::make_series(42);                      // scalar int
auto c = pluss::make_series(true);                    // scalar bool
auto d = pluss::make_series(3.14, ScalarType::Float64);
```

3.2.2 2) 1D/2D/N-D from literals

```
auto v = pluss::make_series({1.0, 2.0, 3.0});
auto m = pluss::make_series({{1.0, 2.0}, {3.0, 4.0}});
auto t = pluss::make_series({{{1,2},{3,4}}, {{5,6},{7,8}}});
```

3.2.3 3) From lvalue host buffers

```
std::vector<double> flat{1,2,3,4,5,6};
auto x = pluss::make_series(flat, {2,3});
```

3.2.4 4) Shape factories

```
auto e = pluss::empty({2,3}, ScalarType::Float32);
auto z = pluss::zeros({2,3});
auto o = pluss::ones({2,3});
auto f = pluss::full({2,3}, 7.5);
```

3.2.5 5) Range and random factories

```
auto a = pluss::arange(0, 10);
auto b = pluss::linspace(0.0, 1.0, 11);
auto c = pluss::randn({3,4});
auto d = pluss::randint(0, 100, {2,3});
auto e = pluss::normal(0.0, 1.0, {5});
```

3.2.6 6) Like-factories

```
auto base = pluss::zeros({2,3}, ScalarType::Float64);
auto a = pluss::zeros_like(base);
auto b = pluss::rand_like(base, ScalarType::Float64, 123);
auto c = pluss::eye_like(base);
```

3.2.7 7) Complex factories

```
auto re = pluss::linspace(0.0, 1.0, 4);
auto im = pluss::linspace(1.0, 2.0, 4);
auto z1 = pluss::complex(re, im, ScalarType::Complex64);
auto z2 = pluss::polar(re.abs(), im, ScalarType::Complex64);
```

3.3 Core Usage Patterns

3.3.1 Inspect and move data

```
auto x = pluss::make_series({1.0,2.0,3.0});
std::cout << x.dtype() << " " << x.sizes() << "\n";

auto cpu_f32 = x.to(pluss::Platform::CPU, pluss::ScalarType::Float32);
auto host = cpu_f32.to_host<float>();
```

3.3.2 Math and reductions

```
auto x = pluss::normal(0.0, 1.0, {20});
auto y = x.abs().add(1.0).log();
auto mean = y.mean();
auto stdv = y.std(false);
auto s = y.sum();
```

3.3.3 Indexing/shape transforms

```
auto m = pluss::make_series({1,2,3,4,5,6}, {2,3});
auto t = m.transpose(0,1);
auto r = m.reshape({3,2});
auto f = m.flatten(0,1);
```

3.3.4 Masking and gather/scatter

```
auto src = pluss::make_series({10,20,30,40,50,60}, {2,3});
auto idx = pluss::make_series({2,1,0,0,1,2}, {2,3});
auto g = src.gather(1, idx);

auto base = pluss::zeros({2,3}, ScalarType::Int64);
auto val  = pluss::make_series({1,2,3,4,5,6}, {2,3});
auto out  = base.scatter_add(1, idx, val);
```

3.3.5 Signal/statistics utilities

```
auto x = pluss::make_series(std::vector<double>{-3,-2,-1,0,1,2,3}, {7});
auto xdct = x.dct();
auto xrec = xdct.idct();
```

```cpp
auto h = x.hilbert();
auto hist = x.histcounts(8, -3.0, 3.0);
auto skew = x.skewness();
auto kurt = x.kurtosis();
```

3.4 Series Demo Map

Demo	Best for
`series_usage_demo.cpp`	Core construction, device/dtype, printing, host extraction
`series_methods_demo.cpp`	Broad method families with pass/fail output
`stats_demo.cpp`	Moments, distributions, moving windows, NaN-aware stats
`signal_demo.cpp`	DCT/DST/Hilbert/phase/fftshift workflows

4 Note (CodeN)

4.1 Include and Basic Construction

```cpp
#include <pluss/pluss.h>
// narrower:
#include <pluss/coden/coden.h>
#include <pluss/coden/note_factory.h>

auto s = pluss::make_note("hello");
auto v = pluss::make_note({"a", "b", "c"});
auto m = pluss::make_note({{"a","b"}, {"c","d"}});
```

4.2 Factory Patterns You Will Actually Use

```cpp
std::vector<std::string> flat{"p","q","r","s"};
auto n = pluss::make_note(flat, std::array<std::int64_t,2>{2,2});

auto from_lines = pluss::note_from_lines("line1\nline2\nline3");
auto from_words = pluss::note_from_words("alpha beta gamma");

auto filled = pluss::full_note({2,1,2}, "z");
auto empty  = pluss::empty_note({2,2});

// raw bytes mode
auto bytes = pluss::bytes_content({{"\x01","\x02"}, {"\x03","\x04"}});
```

4.3 Common Workflows

4.3.1 Text cleanup + normalization

```cpp
auto docs = pluss::make_note("One.\nTwo!");
auto clean = docs.lower().collapse_whitespace().strip();
```

4.3.2 Tokenization and split

```cpp
auto docs = pluss::make_note("alpha beta gamma");
auto words = docs.tokenize_words();
for (std::int64_t i = 0; i < words.tokens().numel(); ++i) {
  std::cout << words.tokens().at(i) << "\n";
}
```

4.3.3 NLP/TeX utilities

```cpp
auto n = pluss::make_note("Email bob@example.com at 9:30am on 2025-01-03")
    ;
auto masked = n.nlp_mask_pii();

auto tex = pluss::make_note("\\textbf{Bold} and $x+y$");
auto txt = tex.tex_to_text();
```

4.3.4 Printing control

```cpp
pluss::NotePrintOptions opt;
opt.max_cell_chars = 24;
opt.max_elements = 10;
opt.edge_items = 2;
std::cout << pluss::print_note(docs, opt) << "\n";
```

4.4 Note Demo Map

Demo	Best for
note_factory_demo.cpp	All construction/factory paths
note_base_methods_demo.cpp	Shape/index/iterator/base helpers
note_method_families_demo.cpp	NLP/TeX/token method families
note_printing_demo.cpp	Display and bytes rendering options

5 Timeline and Duration (CodeH)

5.1 Construct Timeline

```cpp
#include <pluss/pluss.h>
// narrower:
#include <pluss/codeh/codeh.h>
#include <pluss/codeh/timeline_factory.h>

auto t0 = pluss::make_timeline("2024-01-01T00:00:00Z");
auto t1 = pluss::make_timeline({"2024-01-01T00:00:00Z", "2024-01-01T01
    :00:00Z"});
auto t2 = pluss::make_timeline({{"2024-02-01T00:00:00Z", "2024-02-01T01
    :00:00Z"}});

```

```cpp
auto t3 = pluss::from_epoch_nanos(std::vector<std::int64_t>{0, 1'000'000'000}, std::vector<std::int64_t>{2});
```

5.2 Construct Duration and do date arithmetic

```cpp
auto d1 = pluss::duration_seconds(30);
auto d2 = pluss::duration_minutes(5);
auto d3 = pluss::duration_hours(1);
auto d4 = pluss::duration_days(1);

auto t = pluss::make_timeline("2024-01-01T00:00:00Z");
auto t_plus = t + d2;
auto diff = t_plus - t;   // Duration
```

5.3 Datetime field extraction, formatting, timezone

```cpp
auto t = pluss::make_timeline({"2024-01-01T00:00:00Z", "2024-06-15T12:30:00Z"});
auto years = t.year();
auto months = t.month();
auto weekdays = t.weekday();

auto text = t.strftime("%Y-%m-%d %H:%M:%S");
auto local = t.tz_convert("UTC");
auto floored = t.floor("D");
```

5.4 Timeline Demo Map

Demo	Best for
`timeline_non_generated_and_factory_methods_demo.cpp`	Factories, legacy, non-generated helpers + conversions
`timeline_printing_2d_3d_demo.cpp`	2D/3D display and print options

6 TimeIndex (CodeH)

6.1 Construct and inspect

```cpp
#include <pluss/pluss.h>
#include <pluss/codeh/timeindex_factory.h>

auto idx = pluss::make_timeindex({
  "2024-01-01T00:00:00Z",
  "2024-01-01T00:15:00Z",
  "2024-01-01T00:45:00Z",
  "2024-01-01T01:05:00Z",
});

std::cout << idx.freq() << "\n";
std::cout << idx.is_monotonic_increasing() << "\n";
```

6.2 Resample and rolling

```cpp
auto values = pluss::make_series(std::vector<double>{10,20,30,40}, {4});

auto hourly_sum = idx.resample_agg(values, "H", "sum", 1);
auto hourly_mean = idx.resample_agg(values, "H", "mean", 1);

auto win = pluss::duration_minutes(45);
auto roll_sum = idx.rolling_agg(values, win, "sum");
auto roll_max = idx.rolling_agg(values, win, "max");
```

6.3 Slicing, asof, set-like ops

```cpp
auto k = pluss::make_timeline("2024-01-01T00:40:00Z");
auto p = idx.searchsorted(k, false);
auto a = idx.asof(k);
auto s = idx.slice_time("2024-01-01T00:00:00Z", "2024-01-01T01:00:00Z");
```

6.4 TimeIndex Demo Map

Demo	Best for
`timeline_dataframe_timeindex_methods_demo.cpp`	Timeline, dataframe, asof, resample, rolling aggregation

7 Method Families (Quick Reference)

7.1 Series

- Core tensor ops: shape/view/permute/select/slice
- Numeric ops: arithmetic, unary math, comparison, logical, bitwise
- Reductions: sum/mean/min/max/var/std/norm/all/any
- Data movement: to(platform), to(dtype), contiguous, clone, to_host
- Indexing/masking: where, take, gather/scatter/scatter_add/scatter_reduce, masked_select
- Stats/signal extras: pdf/cdf/histcounts/skewness/kurtosis/dct/dst/hilbert

7.2 Note

- Text normalization/cleanup, regex, replace/remove families
- Tokenization families: split/tokenize/ngrams/extractors
- NLP families: masking, stopwords, language/readability/sentiment helpers
- TeX families: strip/normalize/extract/convert helpers

7.3 Timeline/TimeIndex

- Timeline: datetime field extraction, predicates, tz, rounding, format, min/max/sort/unique
- TimeIndex: monotonic/unique checks, search/slice/asof, resample, rolling, set operations

8 Exhaustive API Index Files

These files are generated from code and are intended as searchable reference inventories:

- `docs/generated/series_methods_generated.txt`
- `docs/generated/note_methods_decls_from_code.h`
- `docs/generated/timeline_methods_signatures_from_code.h`
- `docs/generated/timeindex_methods_signatures_from_code.h`

9 Sphinx Migration Note

For long-term user-facing docs, Sphinx is a better fit than a single LaTeX file:

- Multi-page structure with navigation/search
- Better API reference layering (tutorials + reference + examples)
- Easier incremental updates as methods evolve

A starter Sphinx skeleton is included under `docs/sphinx/`.

10 Build This PDF

```
pdflatex -output-directory docs docs/PLUSSMA_USER_MANUAL.tex
pdflatex -output-directory docs docs/PLUSSMA_USER_MANUAL.tex
```